Lab 6: Code and Branch Coverage Analysis

Name: Aman Priyansh
SRN: PES1UG24AM030
Section: A
Repository: https://github.com/AMAN-PRIYANSH/L6_L7_30_vending_machine
Date: 28 October 2026

1. Aim

The aim of this lab was to measure how much of a small Python program is actually run by its tests, using the coverage tool, and then to keep adding tests until at least 90% line coverage and at least 90% branch coverage were reached. A second goal was to understand why line coverage and branch coverage can give different answers for the same set of tests.

2. Program under test

The program assigned to me is vending_machine.py. It models a coin-operated vending machine that sells three items: cola (price 150), chips (100) and candy (75). Change is returned using coins of 100, 50, 25, 10 and 5. The file has three functions:

Function	What it does	Possible results
select_item(item, stock, credit)	Checks that the item exists, that it is in stock, and that enough money has been inserted.	"invalid", "sold_out", "insufficient", "ok"
make_change(amount)	Breaks the change into coins, largest coin first.	List of coins, None if the amount cannot be paid, ValueError if negative
purchase(item, stock, credit)	Uses the two functions above. On success it reduces the stock by one and returns the change.	("dispensed", coins) or a failure status with an empty list

3. Tools and commands

Python 3.14.7, pytest 9.2.0 and coverage 7.17.0, run on Windows in the VS Code terminal. All commands were run from inside the lab6 folder.

Line (code) coverage:

```
coverage run -m pytest <test file>
coverage report -m
```

Branch coverage:

```
coverage run --branch -m pytest <test file>
coverage report -m
```

The -m option adds a "Missing" column to the report, listing the exact line numbers that never ran. I used that column to decide which test to write next.

One thing to note when reading the reports: the TOTAL row also counts the test file itself, which is always 100% because every test runs. The row that matters is vending_machine.py, so all the numbers in this report are taken from that row.

4. Starting point

The repository came with a starter test file, test_vending_machine.py, containing two tests: one that selects an item that does not exist ("water"), and one that asks for zero change. Running line coverage on it gave:

File	Statements	Missed	Cover	Missing lines
vending_machine.py	30	17	43%	8-12, 17, 21-22, 24, 29-36

So more than half of the program had never been run by any test. Matching the missing line numbers to the code:

- Lines 8-12 are the sold out, insufficient credit and "ok" checks in select_item. Only the invalid item case had been tested.
- Line 17 is the error raised for a negative amount.
- Lines 21-22 are inside the coin loop. The only make_change test asked for zero change, so no coin was ever given out.
- Line 24 is the case where the change cannot be made with the available coins.
- Lines 29-36 are the whole purchase function, which was never called.

5. Code (line) coverage

Test file: CodeCoverageTest_AM030_application.py

Most of the missing lines were different return statements, so my approach was simple: write one test for every possible result of every function. If each result is produced once, every line that leads to it has to run.

Test	Input	Expected result	Covers line
test_select_invalid_item	"water"	"invalid"	7
test_select_sold_out	cola, stock 0	"sold_out"	9
test_select_insufficient_credit	cola, credit 100	"insufficient"	11
test_select_ok	cola, credit 150	"ok"	12
test_change_negative_amount_raises_error	-10	ValueError	17
test_change_normal	65	[50, 10, 5]	21-22, 25
test_change_impossible	3	None	24
test_purchase_fails_when_select_fails	cola, stock 0	("sold_out", [])	31
test_purchase_no_exact_change	cola, credit 153	("no_exact_change", []), stock stays 5	34
test_purchase_dispensed	cola, credit 200	("dispensed", [50]), stock 5 to 4	35-36

Two of the purchase tests also check the stock after the call. Coverage does not need this, since the lines run either way. I added it because a machine that reduces its stock without actually giving out the item (or gives it out without reducing the stock) would be a real bug, and only checking the stock catches that.

The no exact change test uses a credit of 153. Cola costs 150, so the change is 3, and since the smallest coin is 5 the machine cannot return it.

Result

File	Statements	Missed	Cover	Missing lines
vending_machine.py	30	0	100%	none

All 10 tests passed and line coverage went from 43% to 100%.

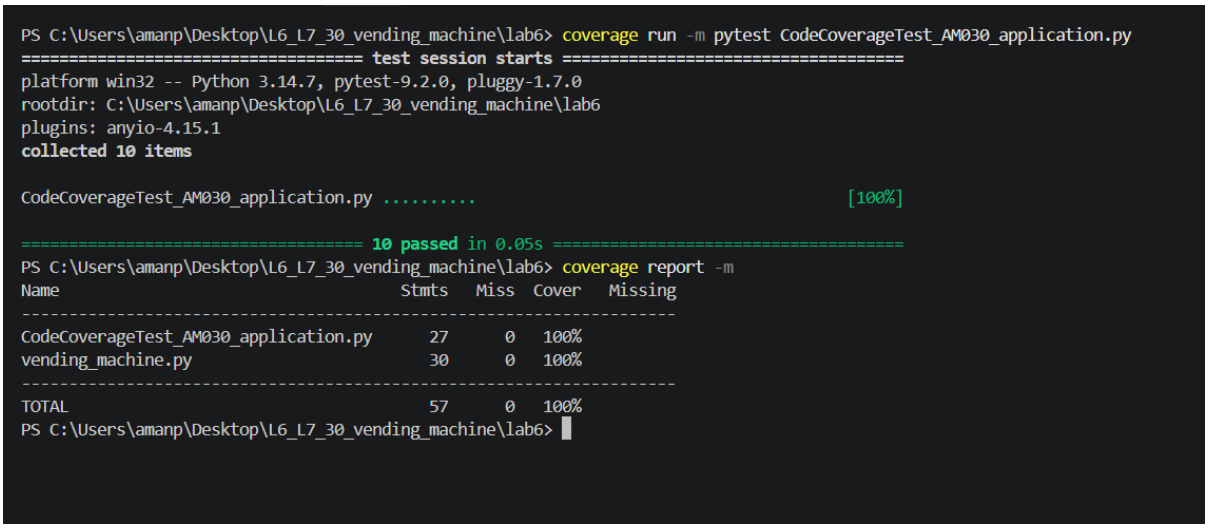


Figure 1: Line coverage report for CodeCoverageTest_AM030_application.py

6. Branch coverage

Test file: BranchCoverageTest_AM030_application.py

A decision point is any place where the program can go two ways: an if, a while, or a for loop. Each of the two ways is called a branch. Line coverage only asks whether every line ran at least once. Branch coverage is stricter: it asks whether every decision was seen going both ways. coverage.py found 9 decision points in vending_machine.py, which gives 18 branches.

For each decision I made sure at least one test sends it each way:

#	Decision (line)	One way, tested by	Other way, tested by
1	item not in PRICES (6)	"water" is rejected	any valid item, e.g. "candy"
2	stock.get(item, 0) == 0 (8)	cola with stock 0; candy missing from the stock dict	cola with stock 5
3	credit < price (10)	cola with credit 149	cola with credit exactly 150
4	amount < 0 (16)	make_change(-1)	make_change(0)
5	for coin in COINS (19)	moves on to the next coin	finishes after the last coin (5)
6	while amount >= coin (20)	make_change(190), make_change(200)	make_change(0), loop never entered
7	amount > 0 after the loop (23)	make_change(3)	make_change(190)
8	status != "ok" (30)	sold out and invalid purchases	valid purchases with enough credit

#	Decision (line)	One way, tested by	Other way, tested by
9	change is None (33)	cola paid with 153	cola paid with 150, chips paid with 175

While writing these I tried to choose inputs that sit right at the edge of each decision, because that is where mistakes usually hide. The ones worth explaining:

- **Credit 149 and credit 150.** These are the values on either side of the cola price. If someone changed credit < price to credit <= price by mistake, a customer paying exactly 150 would be refused. A test with credit 1000 would never notice that, but the 150 test would fail straight away. Values chosen like this are called boundary values.
- **Paying the exact price.** Here the change is an empty list [], not None. The code checks change is None, so an empty list should still go through and the item should be dispensed. The test confirms it does, and that the stock goes from 5 to 4.
- **Candy with an empty stock dictionary.** The code uses stock.get(item, 0), so an item that is not in the dictionary at all should count as sold out. This checks that default value.
- **make_change(0), make_change(190) and make_change(200).** Zero means the while loop is never entered. 190 needs every coin exactly once (100 + 50 + 25 + 10 + 5). 200 needs the same coin twice, which checks that the loop repeats correctly.
- **An invalid item through purchase.** This checks that purchase passes the "invalid" result back correctly, not just "sold_out".

Result

File	Statements	Missed	Branches	Partial	Cover	Missing
vending_machine.py	30	0	18	0	100%	none

All 15 tests passed. The Partial column (BrPart in the terminal) counts decisions that were only ever taken one way. It is 0, which means every one of the 18 branches was taken.

```
PS C:\Users\amanp\Desktop\L6_L7_30_vending_machine\lab6> coverage run --branch -m pytest BranchCoverageTest_AM030_application.py
================================= test session starts =================================
platform win32 -- Python 3.14.7, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\amanp\Desktop\L6_L7_30_vending_machine\lab6
plugins: anyio-4.15.1
collected 15 items

BranchCoverageTest_AM030_application.py ...............                              [100%]

================================= 15 passed in 0.08s ==================================
PS C:\Users\amanp\Desktop\L6_L7_30_vending_machine\lab6> coverage report -m
Name                                      Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------------------
BranchCoverageTest_AM030_application.py      39      0      0      0   100%
vending_machine.py                           30      0     18      0   100%
-------------------------------------------------------------------------------------
TOTAL                                        69      0     18      0   100%
PS C:\Users\amanp\Desktop\L6_L7_30_vending_machine\lab6>
```

Figure 2: Branch coverage report for BranchCoverageTest_AM030_application.py

7. Analysis

7.1 Line coverage and branch coverage on this program

In the calculator example from the handout, 100% line coverage still leaves branches untested, because its if statements have no else part. When the condition is False the program just skips one line and carries on, so a single test where everything is True runs every line but never checks the False side.

The vending machine is written differently. Almost every if ends in a return. To reach any line after an if, that if has to be False, and to reach the return inside it, it has to be True. So a test set that reaches every line is forced to take every decision both ways. When I traced my ten line coverage tests against the nine decisions in the table above, each decision is already taken both ways. For this particular program, reaching 100% line coverage also gives 100% branch coverage.

This comes from the way the code is written (early returns), not from a general rule. The handout example shows that the two numbers can be very different, and branch coverage is the one to trust when they are.

7.2 What the branch tests added

Since the percentage was already 100%, the branch test file did not raise the number. What it added was better test cases: boundary values, exact money, a missing stock entry, change that uses every coin, and change that repeats a coin. This was the most useful thing I learned in this lab. A coverage report can tell me which code was never tested, but it cannot tell me whether the tests that did run were good ones. A test with credit 1000 and a test with credit 150 both cover the same line, but only one of them would catch an off-by-one mistake.

7.3 Problems that full coverage did not catch

While reading the code to write the tests, I noticed some cases that the program does not handle, even though every line and branch is covered:

- **Negative stock.** The check is stock == 0, so a stock count of -1 is treated as available. purchase would then dispense the item and reduce the stock to -2. Using <= 0 would be safer.
- **Decimal amounts.** make_change assumes whole numbers but does not check. An amount of 52.5 would give a 50 coin, leave 2.5 that no coin can pay, and return None. purchase would then report no_exact_change instead of rejecting the bad input.

I did not change vending_machine.py, since this lab is about testing the given code and not fixing it. If the code were updated, these would be the next tests I would add.

8. Commits

Commit	Message	Files added
cc0959f	Code coverage: add test cases (100% line coverage) and screenshot	CodeCoverageTest_AM030_application.py, code_coverage_screenshot.png
c2da4a1	Branch coverage: add test cases (100% branch coverage) and screenshot	BranchCoverageTest_AM030_application.py, branch_coverage_screenshot.png

9. Conclusion

The two starter tests covered only 43% of vending_machine.py. With ten tests, one for each possible result, line coverage reached 100%. The branch coverage file has fifteen tests and reaches 100% branch coverage, with all 18 branches taken and none partial. Both are above the 90% target.

The main takeaway for me is that a high coverage number is necessary but not enough. Coverage shows where tests are missing. Choosing inputs at the edges of each decision is what actually makes the tests catch bugs.